Programming with Generative AI

Viraj Kumar
Kotak-IISc AI-ML Centre

Lecture 4:

- Mathematical and built-in functions
- Simple Python programs

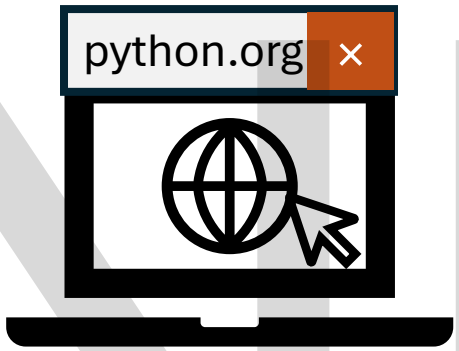


A CSR Initiative by

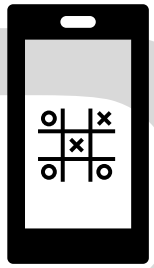


NPTEL

Two Familiar Types of Programs



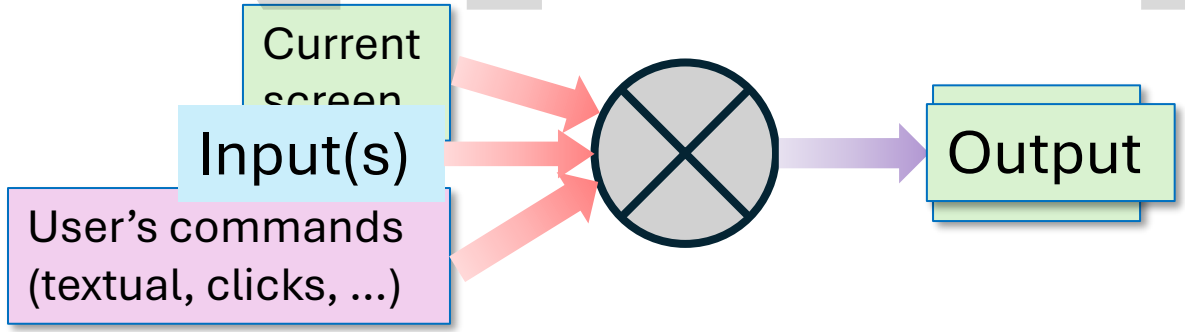
Web apps



Mobile apps

How do they work?

- When we **abstract** the details:



NPTEL

Mathematical Functions

- Recall functions from school Mathematics:

$$f : A \rightarrow B$$

- ***Any*** value $x \in A$ is a valid input to function f, and it must **return** a value $f(x) \in B$ that depends **only** on x
- A **built-in** Python function:

```
>>> ord('')  # Empty string
    TypeError: ord() expected a character,
    but string of length 0 found

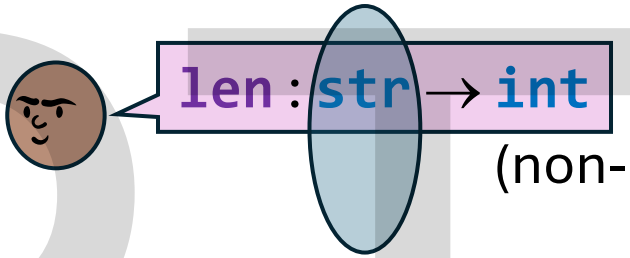
>>> type(type)
    <class 'type'>

>>> type(float)
    <class 'type'>
```

NPTEL

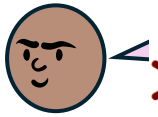
Test Our Friend's Understanding

```
>>> len('')
0
>>> len('1947')
4
>>> len(1947)
TypeError: object of type 'int' has no len()
```



(non-negative)

```
>>> max('1947')
'9'
>>> max('Zero')
'r'
ValueError: max() iterа
```



```
>>> ord('Z')
90
>>> ord('r')
114
>>> ord('z')
122
```

ngth

s empty

Case sensitive

Python's built-in `input` function

```
>>> type(input)
    <class 'builtin_function_or_method'>
```

- Not a mathematical function!

```
>>> input('Name: ')

>>> input('Name: ')

>>> input()
```

NPTEL

Test Our Friend's Understanding

```
>>> temp_in_celsius = input('Celsius: ')
Celsius: -40
>>> 1.8 * temp_in_celsius + 32
TypeError: can't multiply sequence by non-int of type 'float'
```



The `input` function ouputs a `str`, so `temp_in_celsius` is `'-40'` (of type `str`)

Multiplying this `str` with `1.8` (of type `float`) will cause an error

```
>>> 2 * temp_in_celsius
'-40-40'
>>> temp_in_celsius + temp_in_celsius
'-40-40'
```

NPTEL

Let's **debug** our friend's code (*not* in the REPL)

- Each statement is **syntactically** correct

```
1  temp_in_celsius = input('Celsius: ')
2  1.8 * temp_in_celsius + 32
```

- However, there is a **run-time** error on Line 2
  - Let's fix this error on PythonTutor
- Fixed code:

```
1  temp_in_celsius = input('Celsius: ')
2  temp_in_celsius = float(temp_in_celsius)
3  print(1.8 * temp_in_celsius + 32)
```

- Alternative: **nesting**

```
temp_in_celsius = float(input('Celsius: '))
```

str

NPTEL

AI-generated program #1

- Ask ChatGPT to create a similar program:

Write a Python program to convert Celsius to Fahrenheit

```
# Function to convert Celsius to Fahrenheit
def celsius_to_fahrenheit(celsius):
    fahrenheit = (celsius * 9/5) + 32
    return fahrenheit

# Get input from the user
celsius = float(input("Enter temperature in Celsius: "))

# Convert Celsius to Fahrenheit
fahrenheit = celsius_to_fahrenheit(celsius)

# Display the result
print(f"{celsius} Celsius is equal to {fahrenheit} Fahrenheit.")
```

Keywords

User-defined function

Nested function calls

Calling our user-defined function

Each chunk of code starts with a comment explaining that chunk's purpose

Printing with f-strings

Please explain the last line which has a print and an f.

So, for example, if `celsius = 25.0` `fahrenheit = 77.0`, the line will output:

vbnet

Copy

```
25.0 Celsius is equal to 77.0 Fahrenheit.
```



If **celsius** is **25** (of type **int**), why is it being printed as **25.0** (of type **float**)

This is a more concise and readable way to insert variable values into a string compared to the older method of using `str.format()` or string concatenation.

Explanation:

```
# Get input from the user
celsius = float(input("Enter temperature in Celsius: "))
```

bed expressions

d formatted

automatically.

- `{celsius}` and `{fahrenheit}`: Inside the curly braces `{}`, Python evaluates the expressions (variables, in this case) and inserts their values into the string.

NPTEL

AI-generated program #2

- Create a similar program with these tools:
  - IDE: Visual Studio Code
  - Generative AI extension: GitHub Copilot

NPTEL

Summary

- Key building block: **functions**
- Mathematical function $f: A \rightarrow B$ satisfies:
  - $f(x)$ is defined for ***any*** input $x \in A$
  - $f(x) \in B$ and it depends ***only*** on x
- Many useful **built-in** Python functions

```
>>> type(print)
<class 'builtin_function_or_method'>
```

- Function calls can be **nested**
  - Evaluation order: inner to outer
- Don't forget to **critique** AI-generated c[illegible]e!

artefacts

NPTEL